

QUDA 与 PyQCU Strict MultiGrid：算法差异、CUDA 优化与正式对照

PyQCU 工程分析记录

2026 年 9 月 9 日

摘要

本文以 `/root/PyQCU` 中的 PyQCU 和 `/root/PyQCU/refer/git-rep/quda` 中的 QUDA 1.1.0 源码为对象，从 Wilson-Clover 奇偶块矩阵的第一性原理定义出发，逐层解释聚合型 MultiGrid 的 null vector、aggregate、延拓矩阵 P 、限制矩阵 R 、Galerkin 粗算子、粗层 $X/Y/\hat{Y}$ 、Clover/Gauge、MATPC、平滑器、粗层求解器和外层 Krylov 求解器。PyQCU 中同时存在旧的纯 Python/CUDA 33 点 Schur 路径、参考性的 compact Schur 层级和新的 Strict full-coarse 路径；三者在本文中严格分开。本文进一步分析 QUDA 的 transfer、Galerkin、coarse-PC、通信和 solver 优势，定位 PyQCU Strict 的 setup、粗层 kernel、通信与混合精度短板，并记录本轮 device-side MR、cooperative 粗层 BiCGStab、持久状态账本和 block-size 消融。最后复现固定 V100、 $16 \times 32 \times 32 \times 48$ 、c64、odd-odd MATPC 协议，给出五次 steady solve 的总时间、外层迭代数、逐迭代残差和 PyQCU 阶段计时。当前证据支持“同量级并在该正式样本领先”，不支持跨设备或 upstream QUDA 的普遍超越。

目录

1 范围、结论与证据等级	3
2 QUDA 优势、PyQCU 短板与本轮优化	3
2.1 QUDA 的结构性优势	3
2.2 PyQCU Strict 的主要短板	4
2.3 本轮优化与消融	4
3 统一记号、布局 and 量纲	5
3.1 格点、奇偶和自由度	5
3.2 fine Gauge、Clover 和无量纲检查	5
4 三条 PyQCU 路径与 QUDA 路径	6
4.1 旧版纯 Python/CUDA 33 点路径	7
4.2 compact Schur 参考路径	7
4.3 Strict full-coarse 路径	7

目录	2
5 从 fine Schur 到粗层 P/R 和 Galerkin	8
5.1 奇偶 Schur 的完整推导	8
5.2 aggregate、null vector 和 block Gram–Schmidt	8
5.3 Galerkin block 的构造	9
6 粗层 $X/Y/\hat{Y}$、Clover-like dslash 与矩阵顺序	10
6.1 有效粗矩阵的 storage 约定	10
6.2 粗 dslash、粗 Clover 和层间 Gauge 的消失	10
7 奇偶边界：MATPC、prepare 和 reconstruct	11
7.1 full geometry 与 compact parity 的关系	11
7.2 coarse MATPC 的两次 hopping	11
7.3 fine prepare/reconstruct 与 coarse prepare/reconstruct	11
8 平滑器、粗求解器和外层预条件	12
8.1 MR 平滑器	12
8.2 coarsest BiCGStab	12
8.3 V-cycle、W-cycle、F-cycle 与 recursive	13
8.4 右预条件 FGMRES	13
9 一段贯穿全流程的超长公式型伪代码	13
10 QUDA 原生实现的源码级流程	15
10.1 Transfer 与粗化 factory	15
10.2 null-space setup 与递归	15
11 PyQCU Strict 的逐层执行与资产生命周期	16
11.1 setup 的批量构造	16
11.2 Strict C++ 的一次外层迭代	16
11.3 Strict 与 33 点的不可替换边界	16
12 正式大格复现协议与结果	17
12.1 协议	17
12.2 solve-only 总结果	17
12.3 当前 trace 的阶段归因	18
12.4 逐外层迭代 residual	18
12.5 阶段计时：哪些成本主导一次 Strict outer iteration	20

13 Smoke、验证边界和未完成项	20
13.1 已完成的快速回归	20
13.2 QUDA reduction/MultiGrid smoke 的架构阻塞	22
13.3 当前未完成的逐元素核验	23
14 可复现命令、产物和源码索引	23
14.1 命令	23
14.2 本报告直接引用的产物	24
14.3 关键源码索引	24
15 总结：从零复现时应守住的六条不变量	25

1 范围、结论与证据等级

本报告的工作对象是 PyQCU 的 MultiGrid 相关代码，主要对照 `/root/PyQCU/refer/git-rep/quda` 中的 `lib/transfer.cpp`、`lib/dirac_clover.cpp`、`lib/dirac_coarse.cpp`、`lib/multigrid.cpp` 以及粗算子和 transfer CUDA kernel。结论分为三类：

- “源码确证”：可由下文给出的文件和行号直接定位；
- “代数推导”：由已确证的矩阵块和存储约定推出，并给出推导链；
- “未完成的逐元素核验”：不能因为正式求解收敛就冒充为 $Y/\hat{Y}$ 每个方向、每个 parity、每个周期边界位置都已经和 QUDA 逐元素相等。

核心结论可以压缩为如下数据流：

$$(U_0, C_0) \xrightarrow{D_0=A_0-\kappa H_0} V_0 \xrightarrow{R_0 A_0^{\text{eff}} P_0} (X_1, Y_1, X_1^{-1}, \hat{Y}_1) \xrightarrow{V_1} (X_2, Y_2, X_2^{-1}, \hat{Y}_2) \dots \quad (1)$$

这里的 C_0 是 fine Clover， U_0 是 fine Gauge；level 1 以后使用的是有效的矩阵 block，而不是把 SU(3) Gauge “下采样”。一般 Galerkin 关系为

$$D_{\ell+1} = R_\ell A_\ell^{\text{eff}} P_\ell, \quad R_\ell = P_\ell^\dagger, \quad A_\ell^{\text{eff}} \in \{D_\ell, X_\ell^{-1} D_\ell\}. \quad (2)$$

本次正式结果的结论不是“每一次迭代 QUDA 都更快”，而是：在固定的 V100、编译变体、输入 bundle 和计时协议下，PyQCU Strict 的总 solve-only 中位数为 QUDA 的 0.9389 倍，speedup 为 1.065065；它依靠更少的外层迭代抵消了更高的单次外层成本。其他同协议样本覆盖 0.995553–1.102728 倍，故该比值不具有跨平台普适性。

2 QUDA 优势、PyQCU 短板与本轮优化

2.1 QUDA 的结构性优势

- **Transfer 与 P/R** ：QUDA 将 aggregate、spin/chiral map、block orthogonalization、prolongator/restrictor 和 vector tile 固化为专用 CUDA kernel， $R = P^\dagger$ 的存储和通信契约更成熟；

- **Galerkin 与 coarse-PC:** `coarse_op`、`computeYhat` 和 `DiracCoarsePC` 将 UV 、 $V^\dagger UV$ 、 $\text{reverse-}Y$ 、 X^{-1} 、 $\text{forward/backward } \hat{Y}$ 与 `storage conversion` 组织为生产 kernel，能递归扩展到多层；
- **求解器与通信:** QUDA 的 `flexible GCR`、`coarse solver factory`、`cycle` 组织、`QMP/NVSHMEM/-coarse halo` 和硬件调优已经形成统一运行时；
- **工程覆盖:** 多 GPU、多 rank、逐层精度配置、`coarse operator kernel` 和长期硬件回归范围显著大于当前 Strict。

2.2 PyQCU Strict 的主要短板

- `legacy`、`compact Schur` 与 `Strict full-coarse` 三条路径长期并存，语义边界和参数协议更容易被误用；
- Strict 当前对多 rank、非 `nearest-neighbour strict stencil`、奇数 `coarse extent` 和逐层 `mixed precision fail-closed`，尚未具备 QUDA 的分布式生产覆盖；
- `coarse Galerkin setup` 仍保留较多 Python/CUDA tensor 批处理、逐 block 矩阵操作和中间资产，`setup miss` 的算术/内存强度不如 QUDA 专用 kernel；
- 粗层 `hopping`、`dot/reduction` 和 `coarse BiCGStab` 曾有多次 `host/device scalar` 边界，导致单次外层迭代成本高；
- 当前 `formal` 只验证 `c64`、两层、单 rank；没有把逐元素非平凡 X^{-1} 、 Y 、 $\hat{Y}$ 与 QUDA native `coarse dslash` 完整锚定。

2.3 本轮优化与消融

当前 trace 显示 `coarse V-cycle` 中位数为 158.184 ms，其中 `coarsest BiCGStab` 为 151.301 ms，约占一次外层迭代的 85.07%。因此本轮集中优化粗层递推：

1. 将 MR scalar 和向量更新移入 device kernel，减少 host round-trip；
2. 使用 `cooperative-groups` 融合粗层 `hopping`、`dot`、`dot-pair`、`BiCGStab recurrence` 和 `breakdown guard`；
3. 增加持久的两个 `int32` 状态字，并将其计入 `allocated_bytes()` 与测试预算；
4. 通过 `QCU_CUDA_ARCHITECTURES` 提供 native cubin 选择，同时默认保留 60 real + virtual 的旧兼容路径。

上述消融不能替代正式 QUDA 对照；它说明当前最值得继续投入的是粗层 `operator/reduction kernel`、通信重叠和多层 `mixed precision`，而不是把无收益的 `barrier-elision` 留在生产路径。

表 1: 本轮 CUDA 消融; 均为同协议 PyQCU smoke, wall time 只用于方向判断

变体	median solve-only	结论
coarse hopping block 128	2.020994 s	慢于 block 256
coarse hopping block 256	1.926614 s	约快 1.95%, 保留
V100 native sm_{70}	1.964751 s	未优于旧 sm_{60} +PTX 样本
barrier-elision 实验	1.964744 s	无实质收益, 已回退

3 统一记号、布局和量纲

3.1 格点、奇偶和自由度

物理坐标按 (x, y, z, t) 排列, 所有时空轴在 PyQCU 约定中位于最后四维。周期边界下

$$p(x, y, z, t) = (x + y + z + t) \bmod 2, \quad e = 0, \quad o = 1. \quad (3)$$

一次 Wilson hopping 改变一个坐标奇偶, 因此只连接 $e \leftrightarrow o$; Clover 是站点内的 spin-color 矩阵, 不改变 parity。fine Wilson/Clover 的每站点自由度为

$$e = N_s N_c = 4 \times 3 = 12. \quad (4)$$

本次正式协议为 $n_v = 12$ 、coarse spin $N_s^c = 2$, 所以每个 coarse site 拥有

$$E = N_s^c n_v = 2 \times 12 = 24 \quad (5)$$

个粗自由度。这里 n_v 是每一个 coarse-spin block 的 coarse color 数, 不是把 fine spin 直接删掉。

严格路径中, 粗层资产使用完整几何 $[E, X_c, Y_c, Z_c, T_c]$; 只有一个 parity 的场使用 compact 几何 $[E, X_c, Y_c, Z_c, T_c/2]$ 。在 QCU C++ 的 blocked transfer 布局中, fine basis 的静态形状是

$$[E, e, X_c, b_x, Y_c, b_y, Z_c, b_z, T_c, b_t], \quad (X_f, Y_f, Z_f, T_f) = (X_c b_x, Y_c b_y, Z_c b_z, T_c b_t). \quad (6)$$

因此 parity 的“半格”不能被误读为粗资产本身只有半个格点。

3.2 fine Gauge、Clover 和无量纲检查

令 $U_\mu(x)$ 位于 $x \rightarrow x + \hat{\mu}$ 的有向链上, 则未吸收系数的 Wilson hopping 可写为

$$(H\psi)(x) = \sum_{\mu=0}^3 \left[(1 - \gamma_\mu) U_\mu(x) \psi(x + \hat{\mu}) + (1 + \gamma_\mu) U_\mu^\dagger(x - \hat{\mu}) \psi(x - \hat{\mu}) \right]. \quad (7)$$

fine Dirac 算子采用

$$D_0 = A_0 - \kappa H_0, \quad A_0 = \text{diag}(A_{0,e}, A_{0,o}), \quad (8)$$

其中每个 $A_{0,p}(x)$ 是 12×12 Clover onsite block。于是按 (e, o) 排列

$$D_0 = \begin{pmatrix} A_{0,e} & -\kappa H_{eo} \\ -\kappa H_{oe} & A_{0,o} \end{pmatrix}. \quad (9)$$

QUDA 的直接 full operator 在 `refer/git-rep/quda/lib/dirac_clover.cpp:35--65` 调用 `ApplyWilsonClover`, Clover-only 在:50–56, C++ 严格路径则把 `clover_ee`、`clover_oo` 及两个 `inverse` 传进 `cpp/cuda/qcu/src/apply_multigrid_strict.cu:1818--1841`。

格点单位已把晶格间距吸收进参数, U 、 A 、 H 、 D 、 X 、 Y 都是无量纲矩阵; P/R 只改变表示空间, 不改变场的物理量纲。正式配置使用

$$\kappa = \frac{1}{2(m+4)} = \frac{1}{2m+8}, \quad m = 0.05, \quad \frac{1}{2\kappa} = m + 4 = 4.05. \quad (10)$$

最后一个等式也是本次 full residual 归一化中 PyQCU 与 QUDA solution scale 差异的锚点; 它不是 Clover 方向错误。

4 三条 PyQCU 路径与 QUDA 路径

代码库中 “MultiGrid” 不是只有一个类。把下表的路径混为一谈, 会直接把 33 点 Schur stencil、Strict 的最近邻 X/Y 层级和 QUDA 的 native coarse operator 当成同一个 ABI。

表 2: 实现路径的边界和实际算子

路径	层级算子	parity / 粗算子存储	主要求解器与源码
旧版 multigrid	fine parity 可选; 粗层由 <code>dslash.operator</code> 和 33 点 stencil 组成	细层可先做 odd Schur; 粗层 stencil 为 onsite、8 个轴向、24 个双轴位移	每层 BiCGStab, <code>cycle()</code> 在计数达到阈值时做粗修正; <code>pyqcu/solver/_multigrid.py:353--540</code>
QudaMultigrid legacy/compact	支持 full-field Galerkin, 也支持 <code>setup_operator="schur"</code> 的紧凑 parity Schur	compact Schur 的粗自由度作为一个整体; 小问题可 <code>materialize</code> dense, 通用 coarse 有 <code>QudaCoarseOperator/QudaMatPCOperator</code>	可切换 MR/FGMRES 等参考实现; <code>pyqcu/solver/_quda_multigrid.py:909--1968, 1971--2041</code>
Strict	每层先构造完整 coarse 的 X/Y ; 调用边界再做 parity crop	full coarse geometry, 运行期保留 X^{-1} 和 forward/backward $\hat{Y}$; Strict setup 对非最近邻支撑 fail-closed	C++ 持久层级、融合右预条件 FGMRES; <code>cpp/cuda/qcu/src/apply_multigrid_strict.cu:1340--2466</code>
QUDA native	<code>DiracClover/DiracCloverPC</code> 递归到 <code>DiracCoarse/DiracCoarsePC</code>	Transfer full/coarse-site subset 由运行时选择; 粗层 $Y/X/X^{-1}/\hat{Y}$ 是 native GaugeField	MG factory 组合 smoother、coarse solver、 V-cycle/recursive; <code>refer/git-rep/quda/lib/multigrid.cpp:273--430, 564--640</code>

4.1 旧版纯 Python/CUDA 33 点路径

`pyqcu/solver/_multigrid.py:353--540` 的 `cycle()` 对当前层做 BiCGStab; 当 `count_restart` 超过 `num_restart` 且尚有下一层时, 依次执行

$$r \xrightarrow{R_\ell} r_{\ell+1} \xrightarrow{\text{recursive solve}} e_{\ell+1} \xrightarrow{P_\ell} e_\ell, \quad x_\ell \leftarrow x_\ell + e_\ell, \quad r_\ell \leftarrow b_\ell - A_\ell x_\ell. \quad (11)$$

如果 `support_parity=True`, level 0 先用 `give_b_parity` 形成 odd Schur RHS, 求解后由 `give_x_e` 恢复 even 分量。C++ `restrict/prolong` 只在 `with_cuda_qcu` 时替换 Python `einsum`; 粗 dslash 的 packing 为 [2,4,E,E,X,Y,Z,T], 见 `pyqcu/solver/_multigrid.py:91--199`。

`pyqcu/tools/_multigrid.py:277--875` 的 Schur 粗算子不是最近邻 9-block, 而是

$$A_c^{33} = S_c^{(0)} + \sum_{\mu=0}^3 \sum_{\sigma=\pm 1} S_c^{(\sigma\hat{\mu})} + \sum_{0 \leq \mu < \nu \leq 3} \sum_{\sigma, \tau=\pm 1} S_c^{(\sigma\hat{\mu}+\tau\hat{\nu})}. \quad (12)$$

计数为 $1 + 8 + 6 \times 4 = 33$ 。原因是 fine odd Schur 中存在两次 hopping: 两个相反方向给 onsite, 两个同轴方向给 $\pm\hat{\mu}$, 两个不同轴给四种 $\pm\hat{\mu} \pm \hat{\nu}$ 。因此 33 点只适用于已经选定的 Schur 粗化路径; 不能把它直接塞进 Strict 的仅支持 $X + \sum_{\pm\mu} Y_\mu$ ABI。代码中的 PAIRS 和 SIGN 正是 6 对轴和每对 4 个符号组合。

多卡 MultiGpuMultigrid 在 `pyqcu/cuda/_multi_gpu.py:63--168` 为每个线程复制 `params/argv/set_ptrs`, 在主线程或多线程构造 Schur 33 点资产并以 h5py 缓存; worker 线程主要执行各自卡上的 C++ 求解。它要求单 MPI rank, 并且当前 PyTorch wheel 对 P100 sm_60 的 torch kernel 有架构限制, 所以不能拿该路径的 worker 约束替代 Strict 的 full-coarse 语义。

4.2 compact Schur 参考路径

`CompactParityOperator` 将 full field 映射到一个 compact parity, 默认消去 even、暴露 odd Schur:

$$S_o = D_{oo} - D_{oe}A_e^{-1}D_{eo}, \quad b_o^S = b_o - D_{oe}A_e^{-1}b_e. \quad (13)$$

它在 `pyqcu/solver/_quda_multigrid.py:1762--1881` 中显式实现 `apply/rhs/reconstruct`; `QudaMatPCOperator` 则保留完整 coarse 资产, 在应用时通过 checkerboard embed/extract 做两次跨 parity hopping, 见 `pyqcu/solver/_quda_multigrid.py:1539--1595`。这条路径适合小规模代数等价性测试和参考矩阵构造; 它和 Strict 的区别是: Strict 的 X/Y/Yhat setup 必须先在全 coarse geometry 上完成, compact parity 只在 solver 边界出现。

4.3 Strict full-coarse 路径

`QudaMultigrid` 在 `hierarchy_mode="strict"` 时要求 `use_parity=True`, 并在 `pyqcu/solver/_quda_multigrid.py:2089--2227` 拒绝当前未实现的 staggered/KD、奇数 coarse extent 和不一致的 smoother 模式。`CudaStrictMultigridSolver` 在 `pyqcu/cuda/_strict_multigrid.py:90--226` 中完成 MPI preflight、setup、资产绑定、持久 hierarchy 初始化和显存预算; 其运行入口在 `pyqcu/cuda/_strict_multigrid.py:267--334`。

Strict 的重要工程不变量为:

$$\text{applyInitQcu} \rightarrow \text{绑定 level assets} \rightarrow \text{操作} \rightarrow \text{params}[_\text{SET_INDEX_}] += 1 \rightarrow \text{applyEndQcu}. \quad (14)$$

每个实例自己的 `params/argv/set_ptrs` 必须独立; 否则 scratch slot 和 CUDA stream 之间会发生缓冲复用冲突。Strict 的 stage trace 是可选的, 默认关闭, 因此不改变无 trace 生产路径的算子顺序。

5 从 fine Schur 到粗层 P/R 和 Galerkin

5.1 奇偶 Schur 的完整推导

从式 (9) 的第二行得到

$$x_q = A_q^{-1} (b_q + \kappa H_{qp} x_p), \quad q = 1 - p. \quad (15)$$

代入第一行得到非对称 Schur 方程

$$(A_p - \kappa^2 H_{pq} A_q^{-1} H_{qp}) x_p = b_p + \kappa H_{pq} A_q^{-1} b_q. \quad (16)$$

若左乘 A_p^{-1} , 则得到对称归一化 MATPC 算子

$$M_p = A_p^{-1} (A_p - \kappa^2 H_{pq} A_q^{-1} H_{qp}) = I - \kappa^2 A_p^{-1} H_{pq} A_q^{-1} H_{qp}, \quad (17)$$

以及其 RHS

$$b_p^{\text{PC}} = A_p^{-1} (b_p + \kappa H_{pq} A_q^{-1} b_q). \quad (18)$$

这里 p 可以是 even 或 odd; 本次 formal 协议取 $p = \text{o}$ 。QUDA DiracCloverPC::M 在 `refer/git-rep/quda/lib/dirac_clover.cpp:173--210` 分开处理 asymmetric 和 symmetric 分支; `prepare/reconstruct` 在 `refer/git-rep/quda/lib/dirac_clover.cpp:223--260` 直接对应式 (18) 和 (15)。

Strict C++ 的链条是

$$H_{qp} x_p \xrightarrow{A_q^{-1}} A_q^{-1} H_{qp} x_p \xrightarrow{H_{pq}} H_{pq} A_q^{-1} H_{qp} x_p \xrightarrow{A_p^{-1}} A_p^{-1} H_{pq} A_q^{-1} H_{qp} x_p, \quad (19)$$

$$M_p x_p = x_p - \kappa^2 A_p^{-1} H_{pq} A_q^{-1} H_{qp} x_p, \quad (20)$$

对应 `cpp/cuda/qcu/src/apply_multigrid_strict.cu:1843--1863`。如果把目标 parity 的 A_p^{-1} 提前到 RHS 括号内部之前, 得到的将是 $A_p^{-1} b_p + \kappa H_{pq} A_q^{-1} b_q$, 并非式 (18); 该乘法顺序是必须守住的语义边界。

5.2 aggregate、null vector 和 block Gram-Schmidt

给定 block $b = (b_x, b_y, b_z, b_t)$, fine 坐标映射到

$$X_\mu = \lfloor x_\mu / b_\mu \rfloor, \quad \Lambda_{\ell+1} = (L_x / b_x, L_y / b_y, L_z / b_z, L_t / b_t). \quad (21)$$

Wilson/Clover 的 fine spin $4 \rightarrow 2$ 映射为

$$s_c = \lfloor s_f / 2 \rfloor, \quad s_f = 0, 1 \mapsto s_c = 0, \text{quads}_f = 2, 3 \mapsto s_c = 1. \quad (22)$$

每个 coarse site 和 coarse-spin block 内, 对输入的 near-null vectors 做局部 Euclidean block CGS。若 B_j 是第 j 列, 重复两遍时

$$v_j^{(r)} = B_j^{(r)} - \text{sum}_{i < j} v_i^{(r)} \langle v_i^{(r)}, B_j^{(r)} \rangle_{\mathcal{A}}, \quad v_j^{(r)} \leftarrow \frac{v_j^{(r)}}{\sqrt{\langle v_j^{(r)}, v_j^{(r)} \rangle_{\mathcal{A}}}}, \quad r = 1, 2. \quad (23)$$

在 PyQCU 当前参考代码中 $\langle \cdot, \cdot \rangle_{\mathcal{A}}$ 实现为该 aggregate 内的标准复 Euclidean 内积; 符号只强调“局部块内”, 不是另行引入一个未在代码中构造的 Clover 加权内积。实现为 `pyqcu/solver/_quda_multigrid.py:316--346, 446--482`, QUDA 的对应入口为 `Transfer::reset->BlockOrthogonalize`, 见 `refer/git-rep/quda/lib/transfer.cpp:152--163`。

把正交后的基记为 $V_{s_f, c_f, s_c, j}(x)$ 。对 coarse field ϕ 的延拓和对 fine field ψ 的限制为

$$(P\phi)_{s_f, c_f}(x) = \sum_{j=0}^{n_v-1} V_{s_f, c_f, s_c, j}(x) \phi_{s_c, j}(X(x)), \quad (24)$$

$$(R\psi)_{s_c, j}(X) = \sum_{x \in \mathcal{A}_X} \sum_{s_f, c_f} V_{s_f, c_f, s_c, j}(x)^* \psi_{s_f, c_f}(x), \quad (25)$$

$$R = P^\dagger. \quad (26)$$

PyQCU 的 full-field 实现是 `pyqcu/solver/_quda_multigrid.py:538--597, 610--658`, parity view 则在:673–696; QUDA prolongator 先做 geo/spin map 再做 fine-color rotation, 见 `refer/git-rep/quda/include/kernels/prolongator.cuh:61--127`, restrictor 使用共轭基, 见 `refer/git-rep/quda/include/kernels/restrictor.cuh:81--124`。

最重要的两个投影检查是

$$\langle P\phi, \psi \rangle = \langle \phi, R\psi \rangle, \quad P^\dagger P = I_{\text{coarse subspace}}, \quad PP^\dagger \neq I_{\text{fine space}} \quad \text{一般成立}. \quad (27)$$

第二个等式不是说 fine 空间被完整反演, 而是说每个局部正交 coarse basis 在其张成子空间内归一。

5.3 Galerkin block 的构造

对任意有效 fine 算子 A_ℓ^{eff} , 定义

$$D_{\ell+1} = R_\ell A_\ell^{\text{eff}} P_\ell. \quad (28)$$

Strict direct-PC 选择

$$A_\ell^{\text{eff}} = X_\ell^{-1} D_\ell, \quad D_{\ell+1} = R_\ell (X_\ell^{-1} D_\ell) P_\ell. \quad (29)$$

PyQCU 在 `pyqcu/solver/_quda_multigrid.py:2695--2815` 根据 `hierarchy_mode`、`smoother_solve_type` 和 `strict_galerkin_mode` 选择这个算子; batched strict builder 在 `pyqcu/tools/_strict_galerkin.py:594--720` 对每一批 source aggregate 置入 E 个 blocked basis, 调用 batch matvec, 再计算 $V^\dagger A V$ 。

固定 source aggregate X , 只有 source block 和一个 fine nearest-neighbor 所能到达的相邻 aggregate 会非零。令 Δ 属于这些 displacement, 则

$$B_\Delta(X) = \left(P_\ell^\dagger A_\ell^{\text{eff}} P_\ell \right)_{X, X+\Delta}, \quad X_{\ell+1} = B_0, \quad Y_{\ell+1}^{\pm \hat{\mu}} = B_{\pm \hat{\mu}}. \quad (30)$$

Strict builder 的 support check 要求图像在 source aggregate 和八个 one-hop target aggregate 外为零, 见 `pyqcu/tools/_strict_galerkin.py:678--699`。这是 Strict 最近邻 X/Y ABI 的适用条件, 也是 33 点 Schur 路径不能直接复用的原因。

6 粗层 $X/Y/\hat{Y}$ 、Clover-like dslash 与矩阵顺序

6.1 有效粗矩阵的 storage 约定

在目标点 action-coefficient 记号下, 可以写成

$$(D_c z)(X) = X(X)z(X) + \sum_{\mu} \left[Y_{\mu}^f(X)z(X + \hat{\mu}) + Y_{\mu}^b(X)z(X - \hat{\mu}) \right]. \quad (31)$$

QUDA 的 native storage 使用 link-like 约定: forward Y^f 放在源点, backward link 放在 $X - \hat{\mu}$, gather 时取 dagger。对应的 CUDA 注释是 `refer/git-rep/quda/include/kernels/dslash_coarse.cuh:126--141`, backward bulk gather 在:232–249, onsite coarse Clover block 在:257–290。因而不能只对 tensor 做一个 ‘roll’ 就假定方向已经正确, 必须同时确定 storage site、gather site、dagger 和矩阵左右乘顺序。

对 coarse-PC, 令

$$\hat{D}_c = X_c^{-1} D_c = I + \hat{H}_c. \quad (32)$$

需要保存

$$\begin{aligned} \hat{Y}_{\mu}^f(X) &= X_c^{-1}(X) Y_{\mu}^f(X), \\ \hat{Y}_{\mu}^b(X - \hat{\mu}) &= Y_{\mu}^b(X - \hat{\mu}) X_c^{-\dagger}(X - \hat{\mu}). \end{aligned} \quad (33)$$

QUDA 的逐 kernel 证据在 `refer/git-rep/quda/include/kernels/coarse_op_preconditioned.cuh:48--121`: backward 分支先做 $YX^{-\dagger}$, forward 分支做 $X^{-1}Y$; `DiracCoarsePC::createCoarseOp` 在 `refer/git-rep/quda/lib/dirac_coarse.cpp:628--647` 明确说递归粗化的是 $\hat{Y}$ 而不是 Y 。

PyQCU 的 Strict pack 在 `pyqcu/tools/_strict_galerkin.py:489--525` 先批量求 X^{-1} , 将 backward action coefficient 转到 $q - \hat{\mu}$ storage, 再分别计算 `X_inv@forward` 和 `backward_storage@X_inv^{\dagger}`; `QudaCoarseOperator` 的 runtime 绑定在 `pyqcu/solver/_quda_multigrid.py:1265--1305`。因此 $\hat{Y}$ 不是对已经完成的粗 dslash 输出再 “乘一次逆”; 误写为 $X^{-1}Y^b$ 会改变非平凡 Clover 下的算子。

6.2 粗 dslash、粗 Clover 和层间 Gauge 的消失

粗层 dslash 读取的是 $E \times E$ 的有效 link-like block:

$$(D_c z)(X) = X(X)z(X) + \sum_{\mu} \left[Y_{\mu}^f(X)z(X + \hat{\mu}) + Y_{\mu}^{b\dagger}(X - \hat{\mu})z(X - \hat{\mu}) \right], \quad (34)$$

其中把 backward dagger 写在 action 中只是为了与 QUDA 的 storage 对齐。`DiracCoarse::Dslash` 由 `refer/git-rep/quda/include/kernels/dslash_coarse.cuh:140--255` 执行 hop, `applyClover` 执行 onsite block; `DiracCoarsePC::Dslash` 在 `refer/git-rep/quda/lib/dirac_coarse.cpp:506--519` 只把输入 link 换成 $\hat{Y}$, onsite X 仍显式参与。

因此层间对象的类型变化是

$$(U_0 \in SU(3), C_0 \in \mathbb{C}^{12 \times 12}) \longrightarrow (X_1, Y_1 \in \mathbb{C}^{24 \times 24}) \longrightarrow (X_2, Y_2 \in \mathbb{C}^{E_2 \times E_2}). \quad (35)$$

原始 Gauge/Clover 只定义 fine operator 和第一个投影; 后续层的 “Clover” 是 X_{ℓ} , “Gauge” 是 Y_{ℓ} 或 $\hat{Y}_{\ell}$ 的命名类比, 而不是 $SU(3)$ 矩阵。QUDA 的 `DiracCloverPC::createCoarseOp` 入口见

refer/git-rep/quda/lib/dirac_clover.cpp:263--270, 粗层再由 DiracCoarse/DiracCoarsePC 接管。

7 奇偶边界: MATPC、prepare 和 reconstruct

7.1 full geometry 与 compact parity 的关系

每个 coarse level 的资产保留

$$\mathcal{G}_\ell = \{X_\ell, Y_\ell^f, Y_\ell^b, X_\ell^{-1}, \hat{Y}_\ell^f, \hat{Y}_\ell^b\} \quad \text{on full } \Lambda_\ell. \quad (36)$$

compact parity 只定义为 full field 的一个视图。Strict C++ 的 `strict_restrict_parity_kernel` 与 `strict_prolong_parity_kernel` 接收完整 coarse X, Y, Z, T , 在 `cpp/cuda/qcu/src/apply_multigrid_strict.cu:700--820` 处理 parity。若固定前三维, compact 时间索引可以表示为

$$t_{\text{full}} = 2t_c + [(p - x - y - z) \bmod 2]. \quad (37)$$

这解释了为什么“数组第二列”、“物理 odd site”和“checkerboard index”不是同一个概念。当前 Strict 还要求每个非平凡 coarse extent 为偶数, 否则周期 checkerboard 没有等量的两种 parity。

7.2 coarse MATPC 的两次 hopping

用式 (32) 的归一化表示, 目标 parity p 的 coarse MATPC 为

$$M_p^c = I - \hat{H}_{pq} \hat{H}_{qp}, \quad q = 1 - p. \quad (38)$$

严格的应用顺序是

$$u_q = \hat{H}_{qp} x_p, \quad (39)$$

$$v_p = \hat{H}_{pq} u_q, \quad (40)$$

$$M_p^c x_p = x_p - v_p. \quad (41)$$

Strict C++ 在 `cpp/cuda/qcu/src/apply_multigrid_strict.cu:2227--2241` 发射两次 `strict_hopping_parity_kernel`; PyQCU 参考类在 `pyqcu/tools/_strict_galerkin.py:378--404` 先 embed、两次 hopping、再 extract。QUDA 的 odd-odd 分支在 `refer/git-rep/quda/lib/dirac_coarse.cpp:545--557`, 其 full field 会直接报错, 因为 coarse-PC operator 要求 parity subset。

7.3 fine prepare/reconstruct 与 coarse prepare/reconstruct

对 fine 的符号约定 $D = A - \kappa H$, prepare/reconstruct 已由式 (18) 和式 (15) 给出。对粗层, QUDA 把 off-diagonal block 的符号吸收到粗 Y 或 kernel 的 κ 参数中, 所以源码写作

$$b_p^{c, \text{PC}} = X_p^{-1} b_p^c - (X_p^{-1} D_{pq}^c) X_q^{-1} b_q^c, \quad x_q^c = X_q^{-1} b_q^c - (X_q^{-1} D_{qp}^c) x_p^c. \quad (42)$$

若把粗 off-diagonal 定义成 $-D_{pq}^c$, 式中两个减号会变成加号; 代数对象不变, storage 符号必须和 kernel 同时读取。QUDA 的实际分支为 `refer/git-rep/quda/lib/dirac_coarse.cpp:570--625`。

8 平滑器、粗求解器和外层预条件

8.1 MR 平滑器

Strict 的 fine 和 coarse MR 使用同一个非 Hermitian-safe 标量更新。给定当前 residual r ，令 $v = Mr$ ，其中 M 是当前 parity operator，则

$$\alpha = \frac{\langle v, r \rangle}{\langle v, v \rangle}, \quad x \leftarrow x + \alpha r, \quad r \leftarrow r - \alpha v. \quad (43)$$

对应 `cpp/cuda/qcu/src/apply_multigrid_strict.cu:1928--1946` 和 `2283--2300`。分母 finite/breakdown guard 不是装饰：coarse Schur 或方向相关 $\hat{Y}$ 不必在普通 Euclidean 内积下 Hermitian positive definite，因此不能不加条件地替换成 CG。

QUDA 的 smoother factory 在 `refer/git-rep/quda/lib/multigrid.cpp:273--332` 设置 `inv_type`、`tol`、Schwarz cycle、CA basis 以及是否返回 residual；它还分别建立 residual operator、smoother operator 和 sloppy smoother。当前 formal 配置使用 $\nu_{pre} = \nu_{post} = 1$ ，但代码枚举可表达的 MR、CG、CA-CG、GCR、BiCGStab(l) 或 Schwarz 不能被说成在本次正式 run 中全部执行过。

8.2 coarsest BiCGStab

Strict 最粗层使用 BiCGStab；以下表格按用户要求把整轮递推写成单列伪代码，并明确 preconditioned/operator 的位置。令 $A = M_L$ 为最粗层 MATPC，本表中 $M = 1$ 表示不再嵌套第二个 preconditioner。

表 3: *

算法 1: 最粗层 BiCGStab (带可选左/右预处理的单列伪代码)

$r^{(0)} = b - Ax^{(0)}$, $\tilde{r} = r^{(0)}$, p, v, s, t 置零。

for $i = 1, 2, \dots$, 直到 $\|r^{(i)}\|^2 \leq \tau^2 \|b\|^2$:

$\rho_{i-1} = \tilde{r}^\dagger r^{(i-1)}$; 若 $|\rho_{i-1}|$ 或 $|\omega_{i-1}|$ 太小, 则报告 breakdown。

若 $i = 1$, $p^{(1)} = r^{(0)}$; 否则 $\beta_{i-1} = \frac{\rho_{i-1} \alpha_{i-1}}{\rho_{i-2} \omega_{i-1}}$, $p^{(i)} = r^{(i-1)} + \beta_{i-1}(p^{(i-1)} - \omega_{i-1}v^{(i-1)})$ 。

solve $M\hat{p} = p^{(i)}$; 若无预处理, 取 $M = I$, $\hat{p} = p^{(i)}$ 。

$v^{(i)} = A\hat{p}$, $\alpha_i = \frac{\rho_{i-1}}{\tilde{r}^\dagger v^{(i)}}$ 。

$s = r^{(i-1)} - \alpha_i v^{(i)}$; 若 $\|s\| \leq \tau \|b\|$, $x^{(i)} = x^{(i-1)} + \alpha_i \hat{p}$, 然后停止。

solve $M\hat{s} = s$; $t = A\hat{s}$; $\omega_i = \frac{t^\dagger s}{t^\dagger t}$ 。

$r^{(i)} = s - \omega_i t$, $x^{(i)} = x^{(i-1)} + \alpha_i \hat{p} + \omega_i \hat{s}$ 。

若 ρ 、分母或 ω 非 finite, 则保留当前 finite iterate, 重置 shadow/search/scalar 状态或按调用方策略退出。
repeat until convergence or $i = i_{\max}$; 记录 i 、 $\|r^{(i)}\| / \|b\|$ 和耗时。

Strict C++ 的实际最粗层对应 `cpp/cuda/qcu/src/apply_multigrid_strict.cu:2303--2378`；它采用 relative squared stopping target、finite guard 和 $\rho/\alpha/\omega$ 复数递推。legacy Python BiCGStab 还在粗修正后重置整个递推状态，见 `pyqcu/solver/_multigrid.py:487--501`；这是因为 r 已被粗修正改变。

8.3 V-cycle、W-cycle、F-cycle 与 recursive

一次 V-cycle 的误差传播可写成

$$\mathcal{V}_\ell(b) = S_{\ell,post}^{\nu_{post}} \left[S_{\ell,pre}^{\nu_{pre}}(0, b) + P_\ell C_{\ell+1} R_\ell \left(b - A_\ell S_{\ell,pre}^{\nu_{pre}}(0, b) \right) \right]. \quad (44)$$

W-cycle 在同一 child 上调用两次, F-cycle 的第一次下行更强, recursive 则把 child MG object 直接交给 coarse solver wrapper。QUDA 当前 `createCoarseSolver` 的可执行 factory 分支主要是 VCYCLE 或 RECURSIVE, 证据为 `refer/git-rep/quda/lib/multigrid.cpp:564--573`; 枚举中存在 F/W 并不等价于本次 formal 已经运行过 F/W。

Strict 的两层实际执行顺序是:

$$\text{fine MR}_{pre} \rightarrow R \rightarrow \text{level-1 prepare} \rightarrow \text{level-1 BiCGStab} \rightarrow \text{reconstruct} \rightarrow P \rightarrow \text{fine MR}_{post}. \quad (45)$$

C++ 的递归实现 `cpp/cuda/qcu/src/apply_multigrid_strict.cu:2407--2466` 还会在 child 返回后重新计算 parent residual, 而不是保存每一层三份 parent 大小的向量。

8.4 右预条件 FGMRES

Strict formal 外层求解 fine odd Schur, 预条件器是整个 MR-V-cycle-MR:

$$r_0 = b - M_p x_0, \quad z_j = \mathcal{M}_j^{-1} v_j, \quad w_j = M_p z_j, \quad (46)$$

其中 $\mathcal{M}_j^{-1}$ 可因层级、精度、guard 和内部状态而改变, 故使用 flexible 而不是假定固定线性 M^{-1} 的普通 GMRES。Arnoldi/Givens 更新为

$$h_{ij} = \langle v_i, w_j \rangle, \quad w_j \leftarrow w_j - \sum_{i=0}^j h_{ij} v_i, \quad v_{j+1} = w_j / \|w_j\|, \quad x \leftarrow x + \sum_j z_j y_j. \quad (47)$$

Strict C++ 在 `cpp/cuda/qcu/src/apply_multigrid_strict.cu:2070--2215` 实现该递推, 并在 restart boundary 必做 true residual refresh; 正式 run 的 requested restart 为 16, 但 512 MiB workspace 约束把 effective restart 缩到 4。

9 一段贯穿全流程的超长公式型伪代码

下面的单列长表把从输入 Gauge/Clover、null vector 到 full solution 的因果链写在同一处。它同时标出“直接 full Galerkin”、“Schur/33 点”和“Strict/nearest-neighbor”的分叉; 这样从零复现时不会在中途把某一条路径的 parity 或 storage 约定替换到另一条路径。

表 4: 算法 2: PyQCU/QUDA MultiGrid 的 setup-apply-solve 全流程单列伪代码

输入: 周期 fine lattice $\Lambda_0 = (L_x, L_y, L_z, L_t)$ 、Gauge U_0 、Clover $A_{0,e}, A_{0,o}$ 、 κ 、target parity p 、block b_ℓ 、每层 null vector 数 $n_{v,\ell}$ 、coarse spin、smoother、coarse solver、层数 L 、outer tolerance τ 。

按式 (3) 给每个 site 标记 parity; 确认所有用于 checkerboard 的非平凡 extent 为偶数, 并区分 full index、parity index、compact index。

构造 $D_0 = A_0 - \kappa H(U_0)$; Clover 只在 site 内作用, Gauge 只在式 (7) 的 forward/backward hopping 中作用。

若配置 full solve, 设 $q = 1 - p$; 若配置 MATPC solve, 输入输出只保存 p compact field, 但仍保留 q 的 RHS、Clover inverse 和 hopping workspace。

若 null vector 已由 canonical bundle/cache 提供, 读取并验证 shape、dtype、layout、content digest; 否则生成随机/test/inverse/Krylov 起点并以 setup solver 反复平滑, 使 $B \approx D^{-1}$ 的低模方向。

对 $\ell = 0, 1, \dots, L-2$: 按 $X_\mu = \lfloor x_\mu / b_{\ell, \mu} \rfloor$ 建立 fine-to-coarse 与 coarse-to-fine map, 令 $\Lambda_{\ell+1}$ 为 block 后的完整几何。

对 Wilson/Clover 设置 $s_c = \lfloor s_f / 2 \rfloor$; 对每个 aggregate、每个 coarse-spin block, 执行重复 block CGS 和归一化, 得到 $V_\ell(s_f, c_f, s_c, j, x)$ 。

定义 P_ℓ 为 V_ℓ 的 coarse-to-fine contraction, 定义 $R_\ell = P_\ell^\dagger$ 为 fine-to-coarse contraction; 检查 $\langle P\phi, \psi \rangle = \langle \phi, R\psi \rangle$, 但不要把 $PP^\dagger$ 当作 fine identity。

若路径是旧版 Schur: 先在 fine parity 上形成 S_ℓ , 再令 $A_\ell^{\text{eff}} = S_\ell$; 由于两次 hopping, 准备 33 个 displacement $(0, \pm\hat{\mu}, \pm\hat{\mu} \pm \hat{\nu})$ 。

若路径是 full/Strict: 令 $A_\ell^{\text{eff}} = D_\ell$ 或 $X_\ell^{-1} D_\ell$; 在 Strict + direct-PC 中必须使用后者, 不能先投影 D_ℓ 再事后任意补一个 X_ℓ^{-1} 。

对每个 coarse source X 探测 $P_\ell^\dagger A_\ell^{\text{eff}} P_\ell$; 把零 displacement 记为 $X_{\ell+1}$, 把 $+\hat{\mu}/-\hat{\mu}$ 记为 $Y_{\ell+1}^{\text{f/b}}$, 把 Schur 双轴 displacement 记入 33 点 stencil。

若为 Strict, 检查 probe image 在 source aggregate 和八个 one-hop target aggregate 外为零; 若出现双轴或更宽支撑, 拒绝把该算子写入 Strict X/Y ABI。

逐 coarse site 批量求 $X_{\ell+1}^{-1}$; 形成 $\hat{Y}^{\text{f}} = X^{-1} Y^{\text{f}}$, 形成 $\hat{Y}^{\text{b}} = Y^{\text{b}} X^{-\dagger}$, 并把 backward link 放到 $X - \hat{\mu}$ storage; 执行 periodic wrap 和 halo exchange。

丢弃或缓存 raw Y 取决于路径: Strict solve 可只保留 X^{-1} 、 $\hat{Y}$ 、blocked V ; 诊断和后续 coarse construction 可以额外保留 raw Y 。

若 $\ell+1$ 仍需继续粗化, 下一层输入不再是原始 U_0/C_0 , 而是本层有效 (X, Y) 或 coarse-PC 的 $(X, \hat{Y})$; 更新 coarse spin、 n_v 、dof 和几何。

setup 结束后, 构造 residual operator、smoother operator、sloppy operator、presmoother、postsmoother 和 coarse solver; QUDA 中这些对象可以是不同精度或不同 solve type, 不能用一个 “operator” 名称代替全部。

给定 full RHS $b = (b_p, b_q)$ 和初值 x_0 , 先执行 $b_p^{\text{PC}} = A_p^{-1}(b_p + \kappa H_{pq} A_q^{-1} b_q)$; 若只要 MATPC 解则停止在 x_p , 若要 full 解则保留 reconstruct 所需的 b_q 。

设置 $r_0 = b_p^{\text{PC}} - M_p x_{p,0}$; 若 $\|r_0\|$ 已满足容差, 跳过 Krylov 但仍按 full-solution contract reconstruct 并计算 full true residual。

对 outer FGMRES 的每个 restart: 令 $\beta = \|r\|$ 、 $v_0 = r/\beta$, 清零 Hessenberg/Givens/scalar, 并保存 flexible preconditioned basis z_j 。

对 $j = 0, 1, \dots$: 先做 ν_{pre} 次 fine MR; 把当前 residual 通过 R_0 限制到完整 coarse geometry; 调用 child V-cycle/recursive coarse solve。

child solver 内部对非最粗层执行 prepare、pre-MR、restriction、child solve、prolongation、parent residual recompute、post-MR；最粗层执行表 算法 1 的 BiCGStab 或配置的 GCR/CA-GCR。将 coarse correction 通过 P_0 延拓回 fine target parity，做 fine correction residual，再做 ν_{post} 次 MR，得到 $z_j = \mathcal{M}_j^{-1}v_j$ 。

用 $w_j = M_p z_j$ ，计算 $h_{ij} = v_i^\dagger w_j$ ，正交化、归一化并应用 complex Givens；把当前 least-squares estimate 与 basis floor 写入 trace。

若 estimate 满足阈值或发生 happy breakdown，退出 inner cycle；回代小型上三角系统得到 y_j ，并用 $x_p \leftarrow x_p + \sum_j z_j y_j$ 更新解。

在每个 restart boundary 都重新算 $r = b_p^{\text{PC}} - M_p x_p$ ；Strict 中该 true-residual refresh 是强制的，不能只相信递推 residual。

若 $\|r\| > \tau \|b\|$ 且 outer iteration 未达到上限，继续下一个 restart；若收敛，进入 reconstruct。

reconstruct：按 $x_q = A_q^{-1}(b_q + \kappa H_{qp} x_p)$ 恢复 fine 被消去 parity；粗层按其 storage 符号执行对应 X_q^{-1} 和 Y_q 组合。

构造 full $x = (x_p, x_q)$ ，以同一份 periodic Wilson/Clover full operator 计算 $\eta_{\text{true}} = \|b - D_0 x\|_2 / \|b\|_2$ ；只以它作为跨实现的正确性 gate。

保存每次 outer iteration 的 residual kind、wall time、stage time、level、restart boundary、最终 full true residual 和配置 hash；将 timing 和 trace 分开，避免 trace 同步开销污染正式 solve-only 中位数。

10 QUDA 原生实现的源码级流程

10.1 Transfer 与粗化 factory

QUADA 的 Transfer 构造中先校验 aggregate size 与 N_{vec} ，再分配 V、fine/coarse map 和 spin map，见 `refer/git-rep/quda/lib/transfer.cpp:23--114`；`createV` 在:117–149 把 coarse vector 数编码到 native field order，`reset` 在:152–163 调用 block orthogonalization。`createGeoMap` 在:200–230 把每个 fine site 映射到 parity-ordered coarse site。

`MG::createCoarseDirac` 选择用于构造粗算子的 `diracResidual` 或 `diracSmoother`：

$$\text{preconditioned_coarsen} \iff (\text{coarse solution} = \text{MATPC}) \wedge (\text{smoother solve} = \text{DIRECT_PC}). \quad (48)$$

源码在 `refer/git-rep/quda/lib/multigrid.cpp:345--421`，并根据之前层是否已经 preconditioned 决定 `need_bidirectional`。这说明 QUDA 不是单纯“每层重做相同的 dslash”：residual/smoother/PC 的选择会改变粗化输入和必须保留的方向信息。

10.2 null-space setup 与递归

没有从文件加载时，`MG::generateNullVectors` 在 `refer/git-rep/quda/lib/multigrid.cpp:1275--1320` 设置 setup solver、tol、precision、CA basis 和 `compute_null_vector`，然后用 solver 更新 null vector；粗层 null vector 也可由 $R_\ell B_\ell$ 后重新正交得到。本次 formal 对照固定使用已经转换并验证的 12 个 canonical full vectors，所以不能把 formal setup 时间解释为“两侧都在线生成了同样的 null vectors”。

`MG::operator()` 首先按 outer MATPC type 选择 parity, 并按 `coarse_grid_solution_type` 切换 full/parity site subset, 证据为 `refer/git-rep/quda/lib/multigrid.cpp:1131--1160`。若 smoother 和 coarse correction 的 solution type 不一致, QUDA 会显式重算 residual; 这正是 residual operator 与 smoother operator 必须分开的工程原因。

11 PyQCU Strict 的逐层执行与资产生命周期

11.1 setup 的批量构造

Strict setup 的主要步骤是:

1. `QudaTransfer` 生成 full-field V , 并导出 QCU blocked layout;
2. `build_strict_galerkin` 以 colored/site batch 方式计算 $R(X^{-1}D)P$, 检查 nearest-neighbor support;
3. `_finish_strict_galerkin` 生成 (X, X^{-1}) 、raw Y (可选) 和 forward/backward $\hat{Y}$;
4. `qcu_strict_transition_assets` 把各 transition 的四槽资产写进 `set_ptrs`;
5. `applyMultigridStrictInitQcu` 创建持久 hierarchy, 并根据 $(2m+5)B_f+2B_c$ 守住 outer workspace 上限。

形式上, 严格 builder 的 workspace 由一个粗点批次和 E 个 colored probe 共同决定; 正式数据记录了 coarse sites = 49,152、coarse dof = 24、colored column batch = 12、projection site batch = 4、operator calls = 44。这些统计在 `data/strict_vs_quda_formal_20260909_final_current.json`, 不是从旧 Markdown 复制的估计值。

11.2 Strict C++ 的一次外层迭代

`precondition_fine` 的实际阶段在 `cpp/cuda/qcu/src/apply_multigrid_strict.cu:1948--1994`:

$$\begin{aligned} v \xrightarrow{\text{fine MR}_{pre}} u \xrightarrow{R_0} b_1 \xrightarrow{\text{coarse solve}} e_1 \xrightarrow{P_0} u + P_0 e_1 \\ \xrightarrow{M_p} \text{correction residual} \xrightarrow{\text{fine MR}_{post}} z. \end{aligned} \quad (49)$$

`solve_level` 在:2407–2466 处理子层; `apply_matpc/prepare/reconstruct` 在:2227–2281 只通过完整 geometry 的 assets 做 parity kernel。外层 `fgmres_impl` 每个 restart 强制执行 `true_residual_refresh`, 并将 trace stage 围在 CUDA stream sync 之间; 因此 stage 时间包含诊断同步开销, 仅用于归因。

11.3 Strict 与 33 点的不可替换边界

Strict 的输入 operator 是 full nearest-neighbor D 或左预条件后的 $X^{-1}D$, 粗 block 支持为 zero plus eight axis neighbors; 旧版 Schur 路径的输入是 parity Schur, 天然包含二次 hopping 和 33 个位移。若对 Schur 只保存 Strict 的 X/Y , 双轴贡献会被静默丢弃, 所得对象即使在平凡 Gauge 上偶

尔收敛，也不再是原 Galerkin 算子。这是 Strict builder 进行 support check 而不是盲目截断的数学理由。

12 正式大格复现协议与结果

12.1 协议

所有 timing 使用无 trace benchmark 的 steady samples; trace 仅收集逐外层 residual 和 PyQCU stage。正式配置如下。

表 5: 正式 benchmark 固定协议

项目	固定值
硬件	Tesla V100-SXM2-32GB, compute capability 7.0, 单卡单 MPI rank; UUID 前缀 <code>be23...f0a8</code>
软件	PyTorch 2.10.0+cu128, CUDA 12.8, PyQUDA 0.10.54; QUDA 1.1.0 WSL2 DEV87_REDUCE_SYNC 构建
格点与物理	$(L_x, L_y, L_z, L_t) = (16, 32, 32, 48)$, $m = 0.05$, $\kappa = 0.1234567901234568$, seed 42, 周期边界
层级	2 levels, block $(2, 2, 2, 2)$, $n_v = 12$, coarse spin = 2, coarse dof = 24
奇偶	target parity $p = 1$, QUDA QUDA_MATPC_ODD_ODD
外层	restarted right FGMRES/GCR; requested restart = 16, effective restart = 4, max outer iter = 1000, tol = 10^{-6}
平滑/粗求解	$\nu_{pre} = \nu_{post} = 1$; coarse maxiter = 200, coarse tol = 3×10^{-3}
统计	2 次 warmup + 5 次 steady; 报告 median 和 MAD, warmup 不计入
正确性	full periodic Wilson/Clover true residual gate $\eta_{true} < 5 \times 10^{-6}$
身份校验	<code>config_hash=654b7267...332478</code> , <code>bundle_hash=8c866bd0...a6ca3d31</code>

workspace 量纲检查为

$$B_f = 12 \times 16 \times 32 \times 32 \times 24 \times 8 = 37,748,736 \text{ B}, \quad (50)$$

$$B_c = 24 \times 8 \times 16 \times 16 \times 24 \times 8 = 9,437,184 \text{ B}, \quad (51)$$

$$B_{\text{Krylov}} = (2 \times 4 + 5)B_f + 2B_c = 509,607,936 \text{ B} < 512 \text{ MiB}. \quad (52)$$

12.2 solve-only 总结结果

表 6: 五次 steady solve: 外层总迭代、solve-only 时间和 full true residual

侧	steady samples (外层迭代数/秒数)	median (s)	MAD (s)
PyQCU Strict	11/1.968785, 11/1.966795, 11/1.965058, 11/1.965802, 11/1.967481	1.966795	0.000993
QUDA	37/2.077683, 37/2.074447, 37/2.104998, 37/2.094764, 37/2.105811	2.094764	0.011047

表内每格是“外层迭代数 / 秒数”。于是

$$\frac{t_{\text{QUDA}}}{t_{\text{PyQCU}}} = 1.065065, \quad \frac{11}{37} = 0.2973, \quad \frac{178.800}{56.615} = 3.159. \quad (53)$$

PyQCU 的 outer iteration 较少，但单次 outer iteration 成本约为 QUDA 的 3.159 倍；更少的外层迭代使总时间快约 6.5%。这只适用于本节的固定环境。

表 7: 正确性、setup 和显存归因

指标	PyQCU Strict	QUDA	说明
5 次 steady full true residual	3.6013×10^{-7}	7.3030×10^{-7}	都低于 5×10^{-6} gate, 5/5 converged
setup seconds	15.422357	362.053699	PyQCU 为 schema-v2 runtime cache hit; QUDA 建立 native MG runtime
cache restore	13.764082	–	PyQCU 其余 setup non-cache 为 1.658275 s
owned/runtime assets	4,076,863,488 B	未由 PyQUDA 暴露	PyQCU 约 3.797 GiB
fused workspace	509,607,936 B	native 未暴露	PyQCU 约 0.475 GiB
setup device-wide peak	11,219,046,400 B	21,014,843,392 B	设备级采样可能含 native/其他进程
first-solve peak	11,722,362,880 B	24,529,670,144 B	lazy workspace; 不计入 steady timing

12.3 当前 trace 的阶段归因

独立 trace 与 formal JSON 的 `config_hash`、`bundle_hash` 均通过校验。阶段时间通过同一 CUDA stream 前后同步测量，包含同步开销；外层总区间包住内部区间，因此这些百分比不能相加。QUDA 当前公开 `invertQuda/TimeProfile` 接口没有逐 GCR/V-cycle 回调。

表 8: PyQCU Strict 阶段诊断；只用于归因，不替代 formal wall time

阶段	median	median / outer iteration
coarse V-cycle	158.184 ms	88.78%
coarsest BiCGStab	151.301 ms	85.07%
fine restriction	3.307 ms	1.86%
fine pre/post smoother	2.974/2.829 ms	1.67%/1.59%
true residual refresh	3.175 ms	1.76%

该归因支持 device-side scalar、fused coarse BiCGStab 和 coarse block-size 优化的方向；外层 Arnoldi 不是当前主要瓶颈。

12.4 逐外层迭代 residual

下表采用首个 steady solve 的完整曲线；五次 steady 的迭代数一致，完整 250 行逐迭代记录在 `data/strict_trace_detailed_20260909.csv`。PyQCU 是 Arnoldi least-squares estimate, QUDA

是 GCR iterated residual, 两列不是同一个递推标量; 最终判断仍使用 full true residual。

表 9: 首个 steady solve 的每次外层迭代 residual

k	PyQCU Strict	QUDA
0	1.0000000×10^0	1.0000000×10^0
1	9.3650706×10^{-2}	2.6259840×10^{-1}
2	1.6313400×10^{-2}	1.0671660×10^{-1}
3	2.8122286×10^{-3}	5.6699110×10^{-2}
4	6.4573408×10^{-4}	3.2547150×10^{-2}
5	2.1028987×10^{-4}	2.1907580×10^{-2}
6	6.2526138×10^{-5}	1.2670970×10^{-2}
7	2.1975775×10^{-5}	8.4522200×10^{-3}
8	7.4830368×10^{-6}	5.3343160×10^{-3}
9	3.2432135×10^{-6}	3.8731680×10^{-3}
10	1.2322430×10^{-6}	2.4618190×10^{-3}
11	4.0650434×10^{-7}	1.7779130×10^{-3}
12	—	1.1777290×10^{-3}
13	—	8.7806670×10^{-4}
14	—	6.0349930×10^{-4}
15	—	4.3366300×10^{-4}
16	—	3.1065080×10^{-4}
17	—	2.3763540×10^{-4}
18	—	1.6512580×10^{-4}
19	—	1.2454290×10^{-4}
20	—	9.0326790×10^{-5}
21	—	6.8630000×10^{-5}
22	—	5.0507290×10^{-5}
23	—	3.7877360×10^{-5}
24	—	2.8157010×10^{-5}
25	—	2.2275490×10^{-5}
26	—	1.6195150×10^{-5}
27	—	1.2465930×10^{-5}
28	—	9.3341650×10^{-6}
29	—	7.4462140×10^{-6}
30	—	5.4595650×10^{-6}
31	—	4.2656100×10^{-6}
32	—	3.2030910×10^{-6}
33	—	2.5857410×10^{-6}
34	—	1.8816930×10^{-6}
35	—	1.5172770×10^{-6}

k	PyQCU Strict	QUDA
36	–	1.1199800×10^{-6}
37	–	9.1188520×10^{-7}

PyQCU 在第 11 轮的内部 estimate 为 4.0650×10^{-7} ；QUDA 在第 37 轮的 GCR iterated residual 为 9.1189×10^{-7} 。不能对这两列逐点相减后宣称“PyQCU 误差”；可比较的是下降趋势和独立 full-op gate。

12.5 阶段计时：哪些成本主导一次 Strict outer iteration

PyQCU trace 共含 5 个 steady solve、每个 11 个 outer iteration；因为阶段 trace 会在 CUDA stream 前后同步，表中数值含同步开销，并且 nested stage 不能相加。QUDA 当前 `invertQuda/TimeProfile` 公开路径只给 aggregate profile，没有逐 GCR 或逐 V-cycle 的回调，因此下表不伪造 QUDA 阶段值。

表 10: PyQCU Strict 阶段中位数 (55 个 outer iteration 的阶段)

level	stage	count	median (ms)
0	outer iteration	55	185.852
1	coarse V-cycle	55	164.769
1	coarsest BiCGStab	55	157.980
1	level prepare	55	3.247
1	level reconstruct	55	2.923
0	fine pre-smoother	55	3.257
0	fine restriction	55	3.307
0	fine prolongation	55	2.726
0	fine correction residual	55	2.644
0	fine post-smoother	55	3.104
0	fine MATPC (nested)	55	2.476
0	Arnoldi (nested)	55	1.319
0	FGMRES solution update	15	0.630
0	true residual refresh	15	3.090

其中 coarse V-cycle 占 outer iteration 的中位数约 88.9%，coarsest BiCGStab 本身约 85.1%。这给出了比“总时间/总迭代”更有用的热点判断：若要优化当前 formal，首先应分析最粗层递推、粗层矩阵应用和同步，而不是只优化几毫秒的 P/R kernel。

13 Smoke、验证边界和未完成项

13.1 已完成的快速回归

本轮已经运行并通过：结果摘要为 `data/strict_fast_20260909_final.json`；同一源码在按 UUID 隔离的 V100 上另通过 CUDA Strict ‘10’ 与融合 FGMRES ‘3’ 个测试。默认构建采用 60

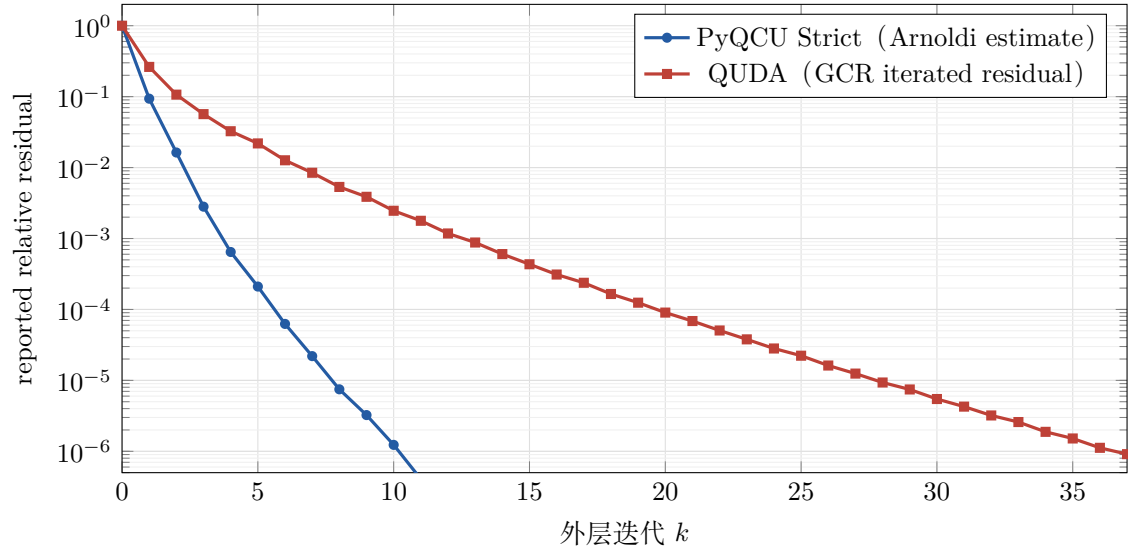


图 1: 首个 steady solve 的逐外层 residual。两条曲线的 scalar contract 不同，图只用于展示各自递推的收敛行为。

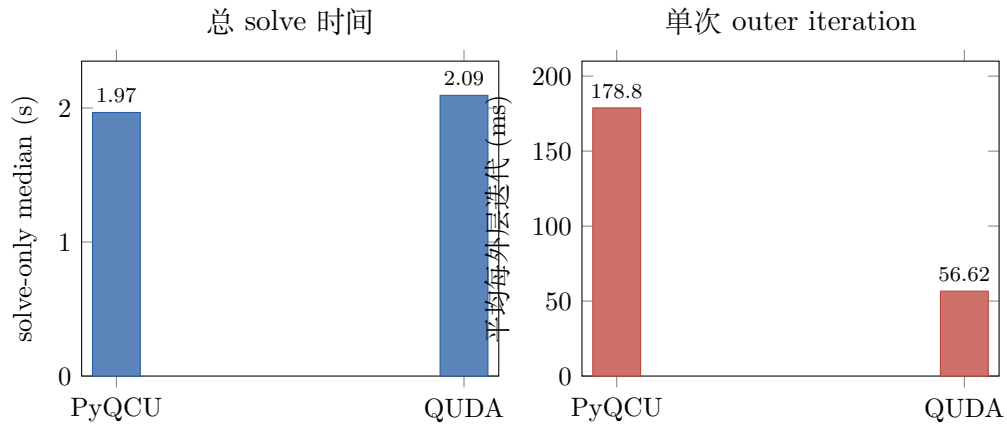


图 2: 总时间和单次外层迭代时间必须同时看；只看后者会错误地忽略迭代数差异。

表 11: 语义与实现快速闸门

类别	数量	结果	覆盖内容
CPU	19	PASS	Transfer、Galerkin、MATPC、prepare/reconstruct、FGMRES 代数
CUDA Strict	10	PASS	full coarse、 $X/Y/\hat{Y}$ 、parity kernel、生命周期
融合 FGMRES	3	PASS	workspace budget、外层收敛、true residual
合计	37	PASS	当前 P100 兼容构建，无 skip/失败

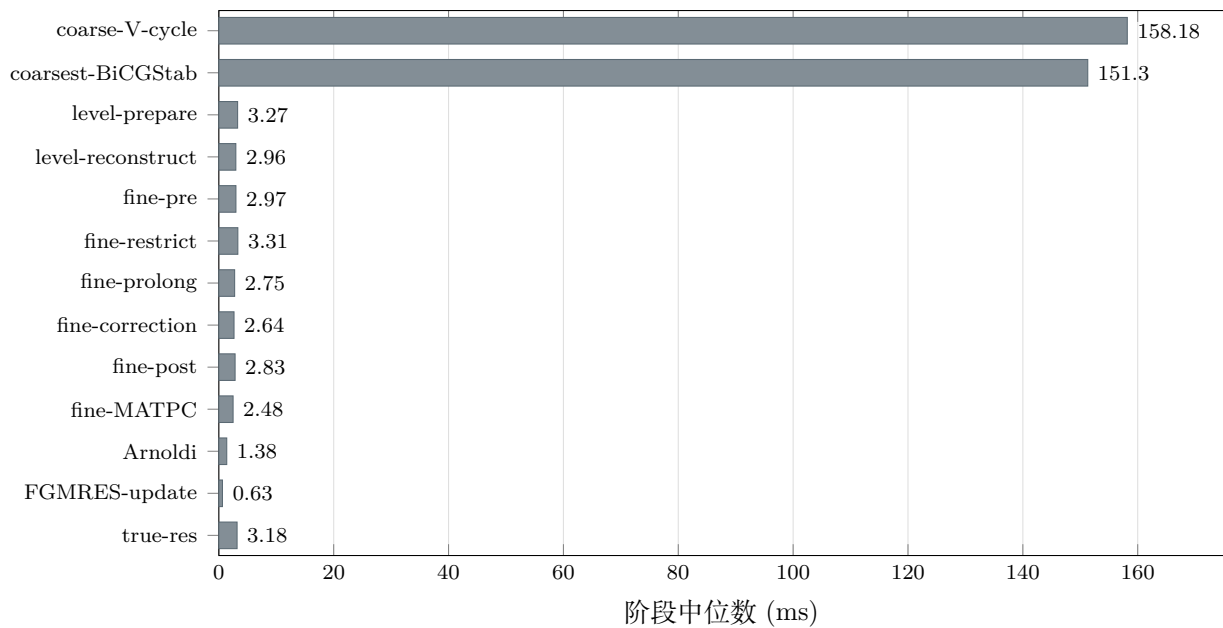


图 3: PyQCU Strict 阶段诊断。嵌套 stage 不可相加；coarse-V-cycle 内含 coarsest-BiCGStab，且所有 stage 均含 trace 同步开销。

real + 60 virtual，显式 70 native 架构构建也均成功。测试不等价于证明 QUDA 与 PyQCU 的每一个 native storage element 相同，但证明了当前 Strict 的闭环可以运行并满足固定协议的 full residual gate。

13.2 QUDA reduction/MultiGrid smoke 的架构阻塞

本轮另外执行了：

```
python -B examples/qcu/dev87/smoke_quda_reduction.py \
  --quda-install data/quda-qio-install
python -B examples/qcu/dev87/smoke_quda_multigrid.py \
  --quda-install data/quda-qio-install
```

两条命令都在 QUDA 第一个 CopyGauge kernel 处报

```
no kernel image is available for execution on the device
QUDA 1.1.0 (git 1.1.0--sm_60)
Using device 0: Tesla V100-SXM2-32GB
```

这表示该 smoke 所选的 libquda.so 是 sm_60 架构构建，而实际设备为 V100 sm_70，且失败发生在进入 MultiGrid setup 之前。因此：

1. smoke 结果应记录为“环境架构阻塞”，不能解释为算法失败；
2. 不能把 smoke 的失败和已经记录的 formal JSON 结果拼成一个“当前所有 QUDA 路径都通过”的结论；
3. 若要重新跑 smoke，应显式选择含 sm_70 SASS/PTX 的 QUDA 安装，并重新记录库 hash、CMake features 和设备架构。

formal JSON 的 QUDA 侧记录了另一份已完成 run 的 provenance; 其中 patch variant 为 dev87 的 WSL2 reduce-sync 构建。它和当前 smoke 的架构选择是两个必须分开的实验事实。

13.3 当前未完成的逐元素核验

仍应优先完成如下实验, 而不应从本报告的 formal convergence 过度推断:

1. 固定同一 canonical nontrivial Gauge/Clover 和同一 V , 导出每个 coarse site、每个方向、两个 parity 的 X^{-1} 、 $Y^{f/b}$ 、 $\hat{Y}^{f/b}$;
2. 对 backward storage 检查 $q - \hat{\mu}$ 的位置、gather dagger、periodic wrap 和 halo boundary;
3. 用同一向量分别计算 QUDA native coarse dslash、PyQCU reference coarse dslash 和 Strict C++ kernel 的逐元素差;
4. 同时比较 $P^\dagger P$ 、RDP、MATPC 两次 hopping 和 full reconstruct, 而不仅比较最终 solver residual。

另外, trace 残差不能替代 true residual; setup hit 的时间不能直接和 online setup miss 比较; 设备级 memory peak 可能包含 QUDA native allocation 或其他进程, 必须和 owned asset bytes 分开报告。

14 可复现命令、产物和源码索引

14.1 命令

```
source ./env.sh
source examples/qcu/dev87/quda_env.sh

# formal solve-only timing (无 trace, 2 warmup + 5 steady)
python -B examples/qcu/dev87/bench_strict_vs_quda.py \
  --profile formal --side both --cache-expect hit \
  --quda-nullvec-prefix data/L16x32x32x48_nvec12_quda \
  --quda-nullvec-manifest data/L16x32x32x48_nvec12_quda.conversion.json \
  --output data/strict_vs_quda_formal_20260909_final_current.json

# diagnostic trace (逐外层 residual 和 PyQCU stage)
python -B examples/qcu/dev87/trace_strict_vs_quda.py \
  --output data/strict_trace_stage_20260909.json \
  --plot data/strict_trace_stage_20260909.svg \
  --benchmark-output data/strict_trace_benchmark_20260909.json \
  --reference data/strict_vs_quda_formal_20260909_final_current.json \
  --repeats 5 --timeout 1800

# 展开 JSON, 生成逐迭代/阶段 CSV、SVG 和摘要
python -B examples/qcu/dev87/analyze_strict_vs_quda_detailed.py
```

14.2 本报告直接引用的产物

- formal: data/strict_vs_quda_formal_20260909_final_current.json;
- trace: data/strict_trace_stage_20260909.json;
- detailed: data/strict_trace_stage_detailed_20260909.json、data/strict_trace_detailed_20260909.csv;
- stage: data/strict_trace_stage_detailed_20260909.json、data/strict_trace_stage_timing_20260909.csv;
- 自动图: data/strict_trace_detailed_20260909.svg;
- 快速回归: data/strict_fast_20260909_final.json;
- 分析器: examples/qcu/dev87/analyze_strict_vs_quda_detailed.py。

14.3 关键源码索引

表 12: 从数学对象回到源码的索引

对象	PyQCU	QUDA
fine $D = A - \kappa H$ / Clover	cpp/cuda/qcu/src/apply_multigrid_strict.cu: 1818--1900	refer/git-rep/quda/lib/di_rac_clover.cpp:35--65
P/R 、spin map、CGS	pyqcu/solver/_quda_multigrid.py:349--597, 610--696	refer/git-rep/quda/lib/transfer.cpp:23--163; refer/git-rep/quda/include/kernels/prolongator.cuh:61--127
full Galerkin / X/Y	pyqcu/tools/_strict_galerkin.py:594--720; pyqcu/solver/_quda_multigrid.py: 909--1086	refer/git-rep/quda/lib/coarse_op.in.cpp:8--20
$X^{-1} / \hat{Y}$	pyqcu/tools/_strict_galerkin.py:489--525	refer/git-rep/quda/include/kernels/coarse_op_preconditioned.cuh:48--121
coarse dslash/Clover	pyqcu/solver/_quda_multigrid.py:1426--1501	refer/git-rep/quda/lib/di_rac_coarse.cpp:506--519; refer/git-rep/quda/include/kernels/dslash_coarse.cuh:126--290

MATPC / compact parity	pyqcu/solver/_quda_multi grid.py:1539--1689,1762 --1881	refer/git-rep/quda/lib/di rac_clover.cpp:173--260; refer/git-rep/quda/lib/di rac_coarse.cpp:530--625
legacy 33 点	pyqcu/tools/_multigrid.p y:277--875; pyqcu/solver/_ multigrid.py:353--540	不属于 QUDA native Strict X/Y ABI; 对应 native coarse 支持见 dslash/coarse-op kernel
Strict recursion/FGMRES	cpp/cuda/qcu/src/apply _multigrid_strict.cu: 1948--2215,2227--2466	refer/git-rep/quda/lib/mu ltigrid.cpp:273--430,564 --640,1131--1160

15 总结：从零复现时应守住的六条不变量

1. 先按式 (9) 写清楚 A 、 H 、 κ 和目标 parity，再讨论 “symmetric/asymmetric”；
2. $R = P^\dagger$ 是 transfer 的代数不变量， P/R 的局部 block map 和 spin map 必须与字段 layout 同时验证；
3. coarse X/Y 是 Galerkin 矩阵 block，coarse “Clover/Gauge” 不是原始 12×12 Clover 和 SU(3) Gauge 的下采样；
4. backward Y 的 storage site、dagger 和 $X^{-\dagger}$ 右乘顺序必须同时正确；
5. full coarse assets 与 compact parity view 分离，MATPC 只在应用、prepare、reconstruct 边界 crop；
6. solver 内部 residual、Arnoldi/GCR reported residual 和 full-op true residual 是三个对象，跨实现正确性只能以最后一个为准。

本报告的正式数据、阶段数据、smoke 限制和未决逐元素核验均已落在工作区；后续若优化 `strict_hopping_parity_kernel` 或改变粗层 storage，应重新生成新的 config hash、formal trace 和 full true residual，而不能用本次结果覆盖新语义。